Notes on the Softmax Function

Luis Mendez

Question: What is softmax, why is it used at the output layer of a multi-class classifier, and how is it trained?

Answer:

1. What Softmax Computes

Given a vector of raw scores (**logits**) $\mathbf{z} = (z_1, \ldots, z_K)$ produced by the last layer of a network, softmax turns them into a probability distribution over K classes:

$$\mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K.$$

Each output $a_k = \mathrm{softmax}(\mathbf{z})_k$ satisfies $a_k > 0$ and $\sum_{k=1}^{K} a_k = 1$, so the vector $\mathbf{a}$ can be read directly as $P(y = k \mid \mathbf{x})$.

2. Why Exponentials

Exponentiating before normalizing does two things a plain ratio $z_k / \sum_j z_j$ would not:

- **Guarantees positivity.** Logits can be negative; e^{z_k} cannot be, so the result is always a valid probability.
- **Exaggerates the largest score.** Because e^z grows faster than z, the class with the biggest logit gets pushed toward probability 1 while the rest are pushed toward 0 — a smooth, differentiable stand-in for $\arg\max$.

3. Relation to Sigmoid

For $K = 2$ classes, softmax collapses to logistic sigmoid. Writing $z = z_1 - z_2$,

$$\mathrm{softmax}(\mathbf{z})_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z).$$

So softmax is exactly the multi-class generalization of sigmoid: sigmoid gives one probability (and its complement); softmax gives K probabilities that all sum to 1.

4. Numerical Stability

Computing e^{z_k} directly overflows for even moderately large logits. Softmax is shift-invariant — subtracting any constant c from every z_k leaves the output unchanged, since c factors out of numerator and denominator:

$$\mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}.$$

In practice $c = \max_j z_j$ is used, so the largest exponent evaluated is $e^0 = 1$ and every other term underflows safely toward 0 instead of overflowing.

5. Loss: Categorical Cross-Entropy

Softmax is almost always paired with **categorical cross-entropy**. For a one-hot true label $\mathbf{y}$ (with $y_k = 1$ for the correct class c and 0 elsewhere),

$$C = -\sum_{k=1}^{K} y_k \log a_k = -\log a_c,$$

i.e. the loss only depends on the probability assigned to the correct class, and it is minimized as $a_c \to 1$.

6. The Gradient Simplifies Beautifully

The Jacobian of softmax alone is a full $K \times K$ matrix,

$$\frac{\partial a_i}{\partial z_j} = a_i(\delta_{ij} - a_j),$$

where δ_{ij} is the Kronecker delta. Multiplying through the chain rule with cross-entropy cancels almost everything, leaving the same clean form as sigmoid + binary cross-entropy:

$$\frac{\partial C}{\partial z_k} = a_k - y_k, \qquad \text{i.e.} \qquad \boldsymbol{\delta}^{(L)} = \mathbf{a} - \mathbf{y}.$$

This is why softmax and cross-entropy are treated as a single combined output layer in practice: the backward pass is just "predicted minus true," with no extra Jacobian to compute or store.

7. Where It Sits in the Network

Softmax is used at the *output* layer, not in hidden layers, because it is defined jointly over an entire vector (each output depends on every logit) rather than elementwise like ReLU or tanh:

$$\mathbf{z}^{(L)} = W^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}, \qquad \mathbf{a}^{(L)} = \mathrm{softmax}\Big(\mathbf{z}^{(L)}\Big) = \hat{\mathbf{y}}.$$

Training then follows ordinary backpropagation and gradient descent, using $\boldsymbol{\delta}^{(L)} = \mathbf{a}^{(L)} - \mathbf{y}$ as the starting error to propagate backward through earlier layers.

8. Putting It Together

- Softmax turns K logits into a probability distribution that sums to 1.
- It generalizes sigmoid from binary to multi-class problems.
- Subtracting $\max_j z_j$ before exponentiating keeps the computation numerically stable.
- Paired with categorical cross-entropy, its gradient reduces to the simple $\mathbf{a} - \mathbf{y}$.

- It belongs at the output layer, feeding directly into the loss.

9. Typical Keras Example

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(m,)),
    Dense(32, activation='relu'),
    Dense(K, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=50, batch_size=32)
```

The final `Dense` layer has one unit per class; Keras applies softmax elementwise across those K logits, and `categorical_crossentropy` implements the loss from Section 5 (use `sparse_categorical_crossentropy` instead if labels are integer class indices rather than one-hot vectors).